Software Requirements Specification (SRS)

Project: CLOZA B2B Multi-Vendor Marketplace

1. Introduction

1.1 Purpose

The purpose of this document is to define the functional and non-functional requirements for the CLOZA B2B multi-vendor marketplace platform, including buyer, vendor, and admin experiences and the financial workflows around Stripe Connect and Klarna deferred payments.

1.2 Scope

CLOZA is a finance-first B2B marketplace that connects business buyers with multiple vendors while CLOZA remains the commercial intermediary. The platform will support:

- Multi-vendor order splitting from a single buyer checkout
- Deferred B2B payments via Klarna, with instant settlement to CLOZA
- Commission-based revenue with dynamic rules and vendor-level payouts
- Advanced admin control, disputes, and analytics.

1.3 Stakeholders

- CLOZA business owners (product and operations)
- Marketplace buyers (B2B customers)
- Marketplace vendors (suppliers)
- CLOZA administrators and finance team.

1.4 Definitions and Abbreviations

- B2B: Business-to-Business
- MOQ: Minimum Order Quantity
- SLA: Service Level Agreement
- KPI: Key Performance Indicator.

2. Overall Description

2.1 Product Perspective

The system is a standalone custom web platform inspired by Shipturtle's multi-vendor philosophy but tailored for B2B and deferred payments. It is not a plugin on Shopify or any existing cart; instead, it uses a custom backend and dashboards.

2.2 User Classes and Characteristics

- **Buyers**:
    - B2B customers placing orders from multiple vendors in a single checkout.
    - Require order history, tracking, and dispute initiation.
- **Vendors**:
    - Businesses listing products and fulfilling orders.
    - Need product management, order management, shipment tracking, and payout visibility.
- **Admins**:
    - CLOZA internal staff managing vendors, products, orders, finance, and disputes.
    - Require full system visibility, override powers, and reporting.

2.3 Operating Environment

- Web application optimized for modern desktop and mobile browsers.
- Backend and database hosted on AWS (RDS, S3, SES) or equivalent cloud infrastructure.

2.4 Design and Implementation Constraints

- Multi-vendor order splitting and vendor-level accounting must support Stripe Connect and Klarna flows.
- Vendors must never see direct buyer contact details.

2.5 Assumptions and Dependencies

- Stripe and Klarna will be available and approved for CLOZA's business model and markets.

- Vendors and buyers will complete KYC/KYB where required by payment providers.

3. System Features

3.1 Buyer Features

3.1.1 Buyer Registration and Authentication

- Buyers can register, log in, and manage their profile and company details.
- Basic KYC/KYB fields may be collected as required by finance rules.

3.1.2 Product Browsing and Search

- Buyers can browse products by category, vendor, and filters (price, MOQ, lead time, etc.).
- Search results show core product info, vendor rating/score, and estimated shipping time.

3.1.3 Cart and Checkout (Multi-Vendor)

- Buyer can add products from multiple vendors into a single cart and place one consolidated order.
- System splits the order internally by vendor while keeping a single reference for the buyer.

3.1.4 Payments with Klarna (Deferred)

- At checkout, buyers can choose Klarna B2B deferred payment options (e.g., 30–60 days) where eligible.
- Buyer sees clear payment terms and due dates; CLOZA receives funds immediately from Klarna.

3.1.5 Order Tracking and History

- Buyers can view current and past orders, including per-vendor shipment statuses.
- Tracking numbers and carrier links are displayed when provided by vendors.

3.1.6 Disputes

- Buyers can open disputes per order or per line item (e.g., damaged, missing, late).
- Buyers can upload evidence (photos, documents) and view dispute status and outcomes.

3.2 Vendor Features

3.2.1 Vendor Onboarding and Approval

- Vendors apply for an account and submit required business and bank details.
- Admin can approve/reject vendors; only approved vendors can list products and receive orders.

3.2.2 Vendor Dashboard

- Vendor dashboard (Next.js + Tailwind UI) shows overview of orders, payouts, and disputes.
- Vendors cannot see buyer contact data; all communication goes through CLOZA.

3.2.3 Product Management

- Vendors can create, edit, archive products through forms and CSV/Excel import.
- Vendors can define MOQ, stock levels, pricing, and expected shipping time per product.

3.2.4 Order Management

- Vendors see only their own orders created from split buyer orders.
- Vendors can accept/confirm orders, mark orders as shipped, and update statuses.

3.2.5 Shipment and Tracking

- Vendors can enter shipment details (carrier, tracking number, ship date).
- System exposes tracking to buyers and admin and updates status where possible.

3.2.6 Earnings and Payouts

- Vendors see gross sales, CLOZA commission, net payout, payment status (pending / paid / on hold), and payout history.
- Vendor view must reflect Stripe Connect balances and payout schedules.

3.2.7 Dispute Handling

- Vendors can see disputes related to their orders, respond, and provide evidence.

- Certain statuses (e.g., payout frozen) are displayed when disputes are open.

3.3 Admin Features

3.3.1 Vendor and Product Management

- Admin can approve/reject vendors and suspend existing vendors.
- Admin can approve/reject products before they become visible, and disable products at any time.

3.3.2 Order and Fulfilment Control

- Admin can view all orders and their split vendor components.
- Admin can override orders: cancel, refund, place on hold, or manually adjust statuses.

3.3.3 Payments and Commissions

- Admin can configure default platform commission and vendor-specific commission rates.
- Admin can monitor Stripe Connect accounts, payout schedules, and freeze/unfreeze payouts during disputes or risk events.

3.3.4 Klarna and Deferred Payments

- Admin can view orders paid via Klarna, see settlement status, and manage exceptions or failures.
- Admin can configure rules around credit limits and deferred payment eligibility (where supported by Klarna APIs).

3.3.5 Disputes and SLAs

- Admin can view and manage all disputes, define statuses, and enforce SLAs for resolution times.
- Admin can decide outcomes (refund, partial refund, vendor penalty, no action) and record notes.

3.3.6 Analytics and Reporting

- Admin dashboard shows KPIs such as: vendor reliability score, dispute rate, average shipping time, revenue per category, top vendors, and payment performance.
- Admin can export key reports (CSV/Excel) for finance and operations.

4. External Interface Requirements

4.1 User Interface

- Responsive web UI for buyers, vendors, and admins using a consistent design system (e.g., Tailwind).
- Separate dashboards/areas for each role with appropriate navigation and permissions.

4.2 API Interfaces

- REST/GraphQL APIs for frontend–backend communication, including authentication, product, order, payment, dispute, and reporting endpoints.
- Secure integration with Stripe Connect and Klarna APIs for payment flows.

4.3 Hardware Interfaces

- No special hardware requirements beyond standard web-capable devices.

4.4 Software Interfaces

- Stripe Connect for marketplace payments and vendor payouts.
- Klarna via Stripe (or direct integration) for deferred B2B payments.
- Email service (AWS SES) for notifications.

5. Non-Functional Requirements

5.1 Security

- Role-based access control: separate permissions for buyers, vendors, admins.
- Protection of financial data and PII according to Stripe/Klarna requirements and best practices.

5.2 Performance

- System should support concurrent use by multiple vendors and buyers without noticeable delay in dashboard operations.
- Order placement and payment processing should complete within acceptable latency targets (e.g., under a few seconds for normal load).

5.3 Reliability and Availability

- High availability for core marketplace functions; target uptime to be defined (e.g., 99.xx%).

- Robust error handling for payment and payout failures, with clear admin and user visibility.

5.4 Scalability

- Architecture must support adding more vendors, products, and orders over time without major redesign.
- Database design must support efficient order splitting, payouts, disputes, and analytics.

5.5 Maintainability

- Modular backend (vendors, buyers, products, orders, shipments, payments, disputes, commissions, reports) to allow incremental changes.
- Clear logging and monitoring to support debugging and operations.

6. Technical Stack

6.1 Frontend

- Next.js + Tailwind for buyer, vendor, and admin dashboards, including product listing, search, order tracking, and dispute UI.

6.2 Backend

- NestJS modular architecture implementing marketplace logic (vendors, buyers, products, orders, shipments, payments, disputes, commissions, reports).

6.3 Database and Infrastructure

- PostgreSQL as the primary relational database for orders, payouts, disputes, and commission rules.
- AWS: RDS for database, S3 for file storage (invoices, labels, proof documents), SES for transactional emails.

7. Timeline and Milestones

Note: *This section is intentionally left for later agreement with the client.*

- **Phase 1 – Discovery & UX Design**
- **Phase 2 – Core Backend & Data Model**
- **Phase 3 – Buyer & Vendor Dashboards (Web)**

- **Phase 4 – Payments (Stripe Connect & Klarna)**
- **Phase 5 – Admin Panel, Analytics & Reporting:**
- **Phase 6 – QA, Security Hardening & Go-Live:**